



CS61C

Announcements

- Additional M1-2pm discussion section (Soda 320)
- Quest Logistics

Great Ideas in Computer Architecture (a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

IEEE 754 Floating Point



Great Idea #1: Abstraction (Levels of Representation/Interpretation)

Week 1	–	Welcome	Number 1: Bit ops, Integer num rep
Week 2	C 1: Intro	C 2: Pointers, Arrays, Strings	C 3: Memory (Stack, Heap)
Week 3	C 4: Odds and Ends	C 5: Generics, Function Pointers	Number 2: Floating Point

Learning C means learning binary abstraction!
Let's revisit **number representation**, but now
for numbers like 4.25, 5.17,

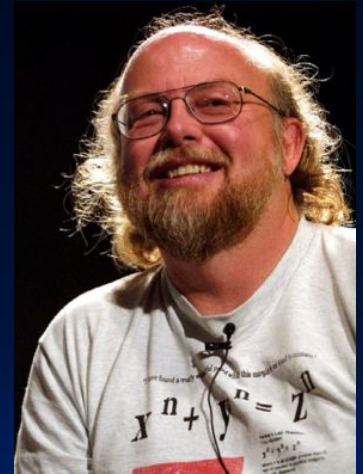


Quote of the Day

"95% of the folks out there are completely **clueless** about floating-point."

– James Gosling, 1998-02-28

- Sun Fellow
- Java Inventor



Yan, SP26

Review of Integer Number Representations

Computers made to process numbers, represented as bits.

What can we represent in N bits?

- 2^N things, and no more! We've covered a few conventions...
- Unsigned integers:
 - Range: 0 to $2^N - 1$
 - N=32: $2^N - 1 = 4,294,967,295$
- Signed Integers (Two's Complement)
 - Range: $-2^{(N-1)}$ to $2^{(N-1)} - 1$
 - N=32: $2^{(N-1)} - 1 = 2,147,483,647$

What about other numbers?

Very large numbers (sec/millennium)

- 31,556,926,010 = $3.155692610 \times 10^{10}$

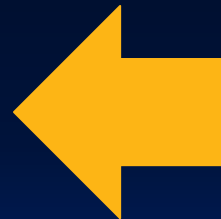
Scientific Notation

Very small numbers? (Bohr radius)

- 0.000000000052917710 = $5.2917710 \times 10^{-11}$

#s with both integer & fractional parts?

- 2.625



Let's start with representing this number...



Agenda

Strawman: Fixed Point

- Strawman: Fixed Point
- Floating Point
- Example
- Special Numbers, Overflow, and Underflow
- Denormalized Numbers

[Reference] FA22 videos: [link](#)

Yan, SP26

"Fixed Point" Binary Representation

"**Binary Point**," like decimal point, signifies the boundary between integer and fractional parts.

Example 6-bit representation, positive numbers only:

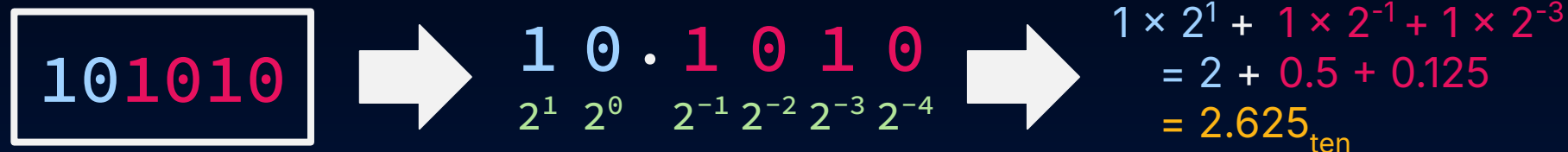




"Fixed Point" Binary Representation

"**Binary Point**," like decimal point, signifies the boundary between integer and fractional parts.

Example 6-bit representation, positive numbers only:



With the above 6-bit "**fixed binary point**" representation:

- Range: 0 to $\underbrace{3.9375}_{11.1111_{\text{two}} = 4 - 1 \times 2^{-4}}$



For Science, but...

Very large numbers (sec/millennium)

- 31,556,926,010 = $3.155692610 \times 10^{10}$

11...0.0
34 bits !

Very small numbers? (Bohr radius)

- 0.000000000052917710 = $5.2917710 \times 10^{-11}$

0.0...11
58 bits !

#s with both integer & fractional parts?

✓ 2.625

To store all these numbers, we'd need a fixed-point rep with at least 92 bits. There must be a better way!

Yan, SP26



Agenda

Floating Point

- Strawman: Fixed Point
- Floating Point
- Example
- Special Numbers, Overflow, and Underflow
- Denormalized Numbers



Floating Point

A “**floating binary point**” most effectively uses of our limited bits (and thus more accuracy in our number representation).

Example:

- Suppose we wanted to represent 0.1640625_{ten} .
- Binary representation: $\dots000000.00\underline{10101}00000\dots$

1. Store these “**significant bits**.”

2. Keep track of the binary point 2 places to the left of the MSb (“**exponent field**”).

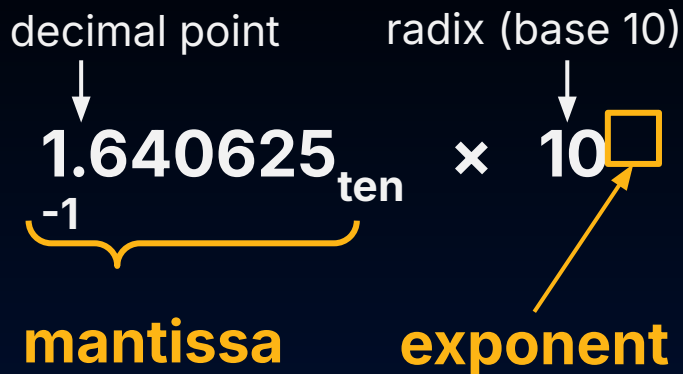
The binary point is stored separately from the significant bits, so very large and small numbers can be represented.



Enter Scientific Notation (in Decimal)

"Normalized form": no leading 0s
(exactly one nonzero digit to left of point).

- To represent $3/1,000,000,000$:
 - **Normalized:** 3.0×10^{-9}
 - Not normalized: 0.3×10^{-8} , 30.0×10^{-10}



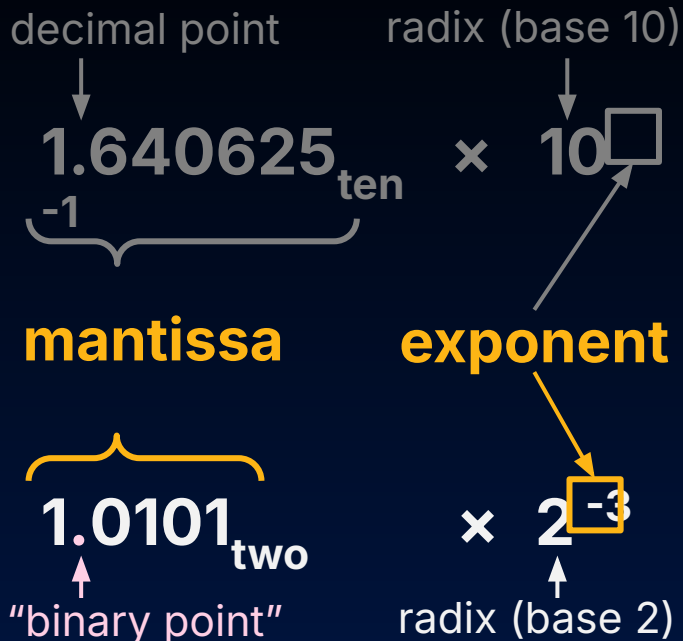
Enter Scientific Notation (in Binary)

"Normalized form": no leading 0s (exactly one nonzero digit to left of point).

- To represent 3/1,000,000,000:
 - **Normalized**: 3.0×10^{-9}
 - Not normalized: 0.3×10^{-8} , 30.0×10^{-10}

"Floating point": Computer arithmetic that supports a binary normalized form.

- Represents numbers where the binary point is not fixed (as opposed to integers).
- C variable types: `float`, `double`



Note: **Normalized** binary representation always leads with a 1! (why?)



31	30		23	22		0
s		exponent		significand		
1 bit		8 bits		23 bits		

$$\pm 1.x_{\text{two}} \times 2^{y_{\text{two}}} \quad \text{Normalized number}$$

- Single precision standard for 32-bit word. In C, `float`.
 - Each field is NOT interpreted as two's complement!

IEEE 754 Single-Precision: Normalized Form

$$(-1)^s \times (1 + \text{significand}) \times 2^{(\text{exponent}-127)}$$

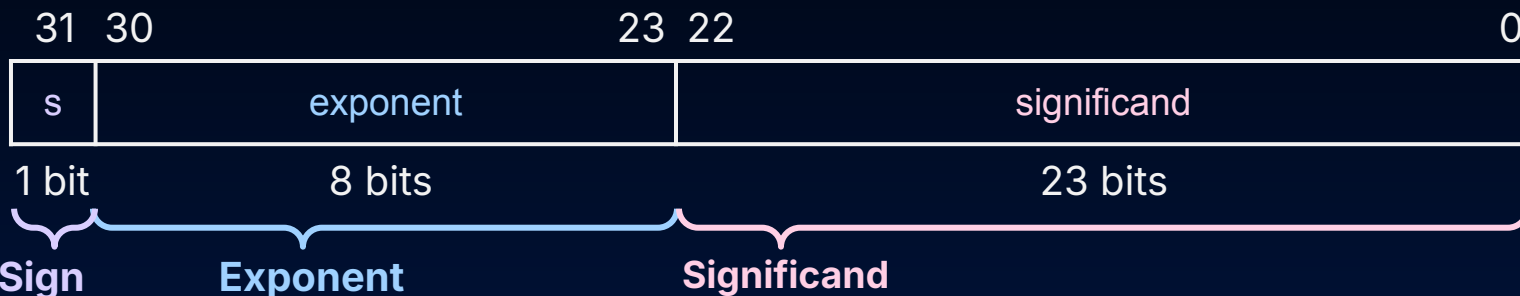


$$\pm 1.\text{xxx}\cdots\text{x}_{\text{two}} \times 2^{\text{yyy}\cdots\text{y}_{\text{two}}}$$

- Instead, designed to maximize **accuracy** of representing many numbers with a limited **precision** of 32 bits.
 - **Precision** is a count of the number of bits used to represent a value.
 - **Accuracy** is the difference between the actual value of a number and its computer representation.

IEEE 754 Single-Precision: Overview

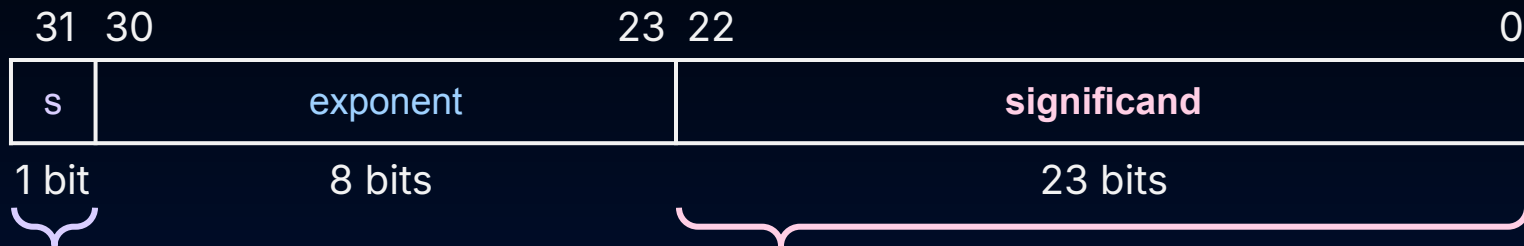
$$(-1)^S \times (1 + \text{significand}) \times 2^{(\text{exponent}-127)}$$



- 1 is negative
- 0 is positive
- Bias of **127**
- Subtract 127 from exponent field to get exponent value
- To pack more bits, mantissa leads w/implicit 1

IEEE 754 Single-Precision Floating Point (2/4)

$$(-1)^s \times (1 + \text{significand}) \times 2^{(\text{exponent}-127)}$$



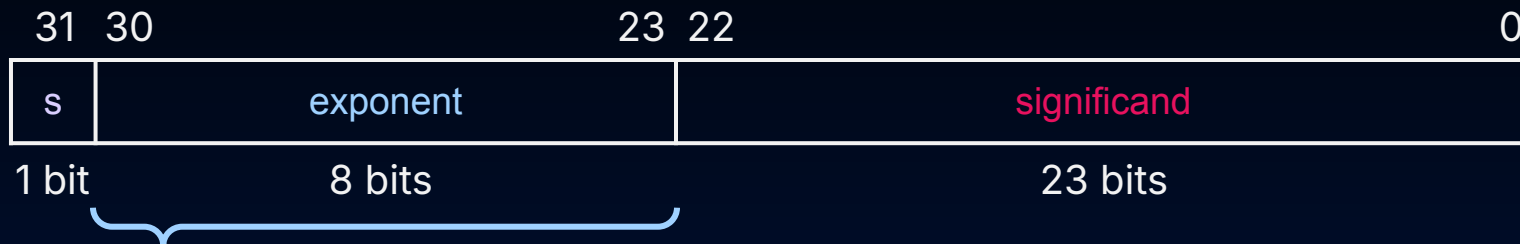
Sign bit:

- 1 is negative
- 0 is positive

Significand field:

- Mantissa = **1** + 23-bit **significand** field
- In normalized, mantissa *always* leads with 1
- Therefore, to pack more bits, IEEE 754:
 - **Treat mantissa leading 1 as implicit**
 - Significand explicitly represents other bits!
 - $0 < \text{significand field} < 1$

IEEE 754 Single-Precision Floating Point (3/4)



Exponent field uses "bias notation":

- **Bias of 127: Subtract 127 from exponent field to get exponent value.**
- Designers wanted FP numbers to be used even without specialized FP hardware, e.g., sort records with FP numbers using integer compares.
- Idea: for the same sign, bigger exponent field should represent bigger numbers.
 - 2's complement poses a problem (because negative numbers look bigger)
 - We'll see numbers ordered EXACTLY as in sign-magnitude:
 - $00 \dots 00 \rightarrow 11 \dots 11$
0 to +MAX $\rightarrow$ -0 to -MAX

$$2^{(\text{exponent}-127)}$$

"Father" of the Floating Point Standard

IEEE Standard 754 for Binary Floating-Point Arithmetic.

**1989
ACM Turing
Award Winner!**

On most systems, C float and double are specified by IEEE 754 floating point format (32b float: single-precision, 64b double precision)



William Kahan
Professor Emeritus
UC Berkeley

www.cs.berkeley.edu/~wkahan/ieee754status/754story.html

08-Floating Point (19)

Yan, SP26



Agenda

Example

- Strawman: Fixed Point
- Floating Point
- Example
- Special Numbers, Overflow, and Underflow
- Denormalized Numbers

Normalized Example

What is the decimal equivalent of the following IEEE 754 single-precision binary floating point number?

31	30		23	22		0
1		1000 0001		111 0000 0000 0000 0000 0000		

$$(-1)^S \times (1 + \text{significand}) \times 2^{(\text{exponent}-127)}$$

- A. $-7 * 2^{129}$
- B. -3.5
- C. -3.75
- D. 7
- E. -7.5
- F. Something else



Yan, SP26

L07: What is the decimal equivalent of the following IEEE 754 single-precision binary floating point number?



A. -7×2^{129}

0

B. -3.5

0

C. -3.75

0

D. 7

0

E. -7.5

0

F. Something else

0



[Solution] Normalized Example

What is the decimal equivalent of the following IEEE 754 single-precision binary floating point number?

31	30		23	22		0
1		1000 0001		111 0000 0000 0000 0000 0000		

- A. $-7 * 2^{129}$
- B. -3.5
- C. -3.75
- D. 7
- E. -7.5
- F. Something else

$$(-1)^S \times (1 + \text{significand}) \times 2^{(\text{exponent}-127)}$$

$$(-1)^1 \times (1 + .111)_{\text{two}} \times 2^{(129-127)}$$

$$-1 \times (1.111)_{\text{two}} \times 2^{(2)}$$

$$-111.1_{\text{two}} \quad -1.111$$

$$-7.5_{\text{ten}}$$



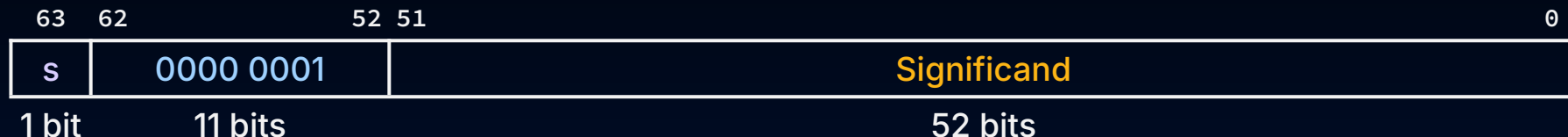
Agenda

IEEE 754 Double-Precision Floating Point

- Strawman: Fixed Point
- Floating Point
- Example
- Special Numbers, Overflow, and Underflow
- Denormalized Numbers

IEEE 754 Double-Precision Floating Point (double)

binary64: Next Multiple of Word Size (64 bits)



Double Precision (vs. Single Precision)

- C variable declared as **double**
- Exponent bias now -1023.
- Represent numbers from $\sim 2.0 \times 10^{-308}$ to $\sim 2.0 \times 10^{308}$.
- The primary advantage is greater accuracy due to larger significand.



Aside: Precision and Accuracy

Precision is a count of the number of bits used to represent a value.

Accuracy is the difference between the actual value of a number and its computer representation.

High precision permits high accuracy but doesn't guarantee it. It is possible to have **high precision** but **low accuracy**.

- Example: `float pi = 3.14;`
- `pi` will be represented using all bits of significand (**precise**), but it is only ever an approximation (less **accurate**).



...caution...



Let's try to find this number!



<https://www.h-schmidt.net/FloatConverter/IEEE754.html>





For you: IEEE-754 Floating Point Converter

4:21 PM

Wed, Jan 22 94%

340,282,346,638,5
28,860,000,000,00
0,000,000,000,000°

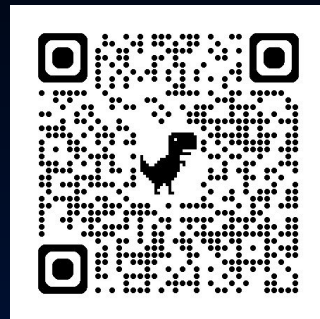
F

Sunny

San Diego, Wed 4:10 PM



[h-schmidt.net/
FloatConverter/
IEEE754.html](https://h-schmidt.net/FloatConverter/IEEE754.html)



What is the floating point representation of this **largest representable number**?

What happens when you set biased exponent 255?



Yan, SP26



Normalized numbers: **biased exponents** are restricted to the range 1 to 254 (2^{-126} to 2^{127}).

Exponent fields 0, 255 are reserved for **special numbers**. More now!



Agenda

Special Numbers, Overflow, and Underflow

- Strawman: Fixed Point
- Floating Point
- Example
- Special Numbers, Overflow, and Underflow
- Denormalized Numbers



Special Numbers

Normalized numbers are only a *fraction* (heh) of floating point representations. For single-precision (32-bit):

Biased Exponent	Significand field	Object
0	all zeros	± 0
0	nonzero	Denormalized number
1 – 254	anything	Normalized floating point
255	all zeros	infinity
255	nonzero	Not a Number

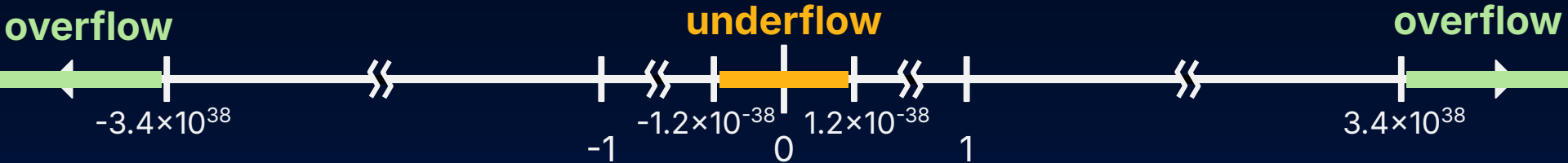
Professor Kahan had clever ideas;
"Waste not, want not."



Overflow and Underflow

Because 0 and 255 are reserved exponent fields, the range of normalized single-precision floating point is limited.

- See course notes for how to compute `FLT_MIN`, `FLT_MAX`



What if a number falls outside the representable range?

- Too large: **overflow**. Magnitude of value is too large to represent!
- Too small: **underflow**. Magnitude of value is too small to represent!



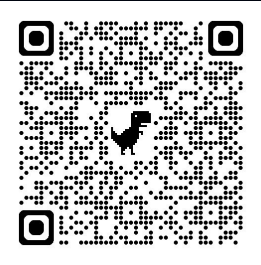
Special Numbers Handle Edge Cases

Unlike integer reps, floats can more “gracefully” handle **overflow**, **underflow**, and **errors** with special numbers (i.e., non-norms).

Biased Exponent	Significand field	Object
0	all zeros	± 0
0	nonzero	Denormalized number
1 – 254	anything	Normalized floating point
255	all zeros	infinity ($\pm\infty$)
255	nonzero	Not a Number (NaN)



Agenda



Denormalized Numbers

Course website
(GitHub) is down.
Access slides on
[EdStem](#) / QR code

- Strawman: Fixed Point
- Floating Point
- Example
- Special Numbers, Overflow, and Underflow
- Denormalized Numbers

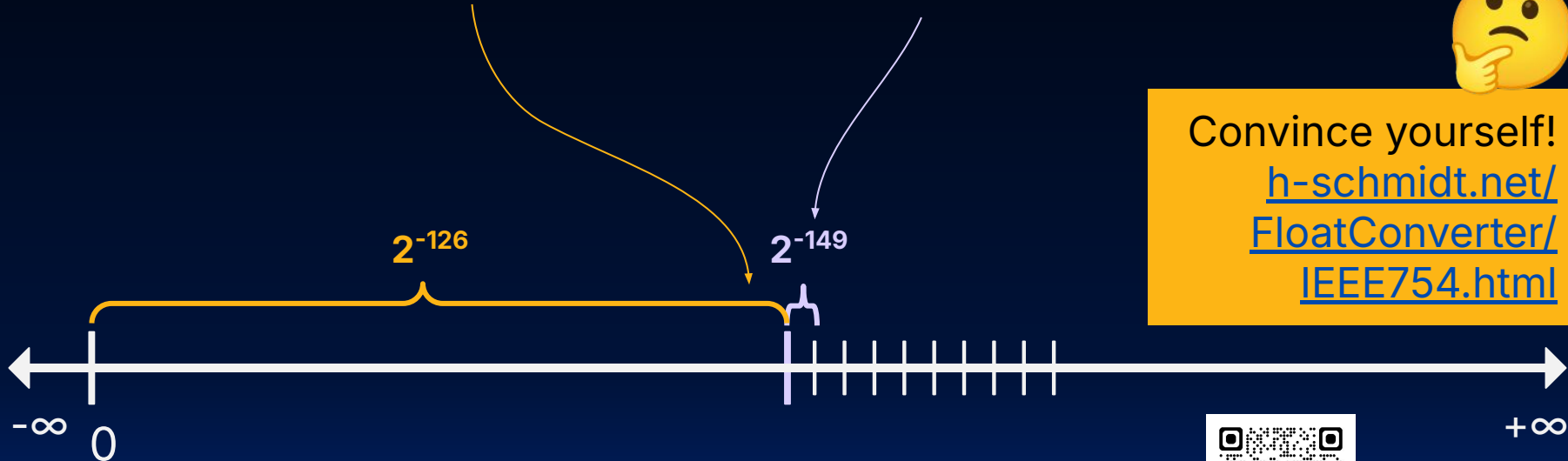
Denorms: Gradual Underflow (1/3)

Problem with normalized/implicit 1:

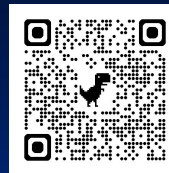
- There's a **gap** among representable FP numbers around zero!

Smallest normalized number:

2nd smallest normalized number:



Convince yourself!
h-schmidt.net/FloatConverter/IEEE754.html



Denorms: Gradual Underflow (1b/3)

Problem with normalized/implicit 1:

- There's a **gap** among representable FP numbers around zero!

Smallest normalized number:

0	0000 0001	000...000
---	-----------	-----------

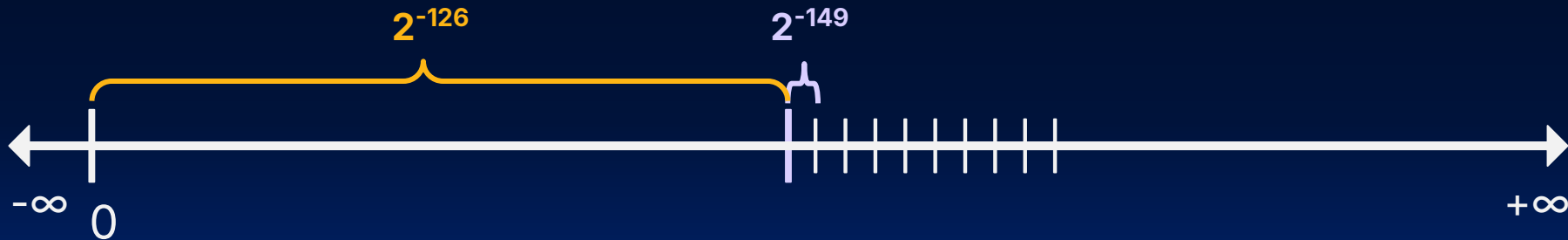
$$(1.0) \times 2^{(1-127)} = 2^{-126}$$

2nd smallest normalized number:

31	30	23	22	1	0
0	0000 0001	00000...00001			

$$(1.000.....1_2) \times 2^{(1-127)}$$

$$= (1 + 2^{-23}) \times 2^{-126} = 2^{-126} + 2^{-149}$$





Denormalized Numbers

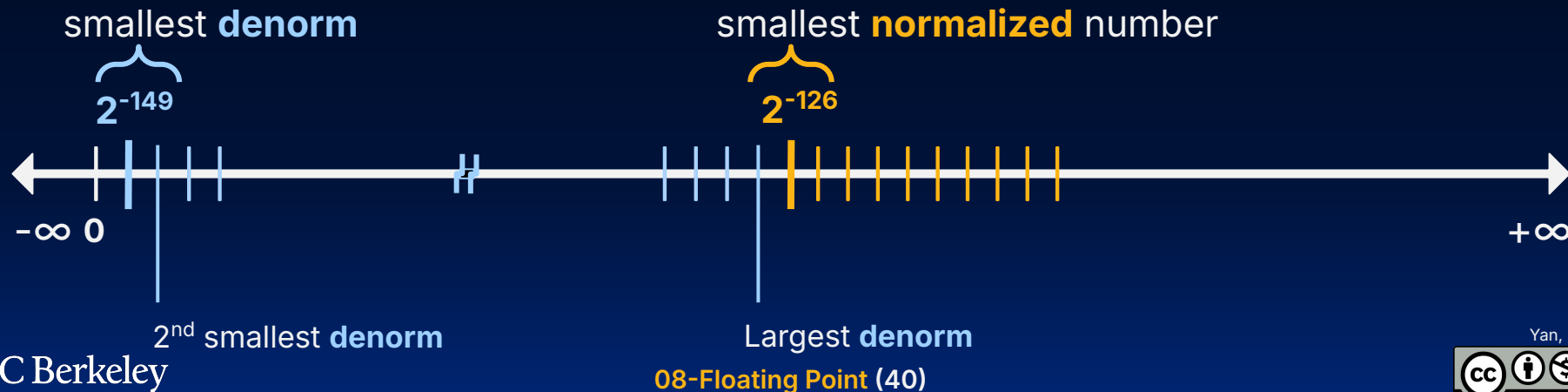
Biased Exponent	Significand field	Object
0	all zeros	± 0
0	nonzero	Denormalized number
1 – 254	anything	Normalized floating point
255	all zeros	infinity
255	nonzero	Not a Number

Denorms: Gradual Underflow (3/3)

- Solution: **DENORMalized number**
 - no leading 1 in **mantissa**!!
 - One **implicit exponent** for all denorms: the smallest norm step size, **2^{-126}**

Denormalized number

$$(-1)^s \times (0.x_1 \dots x_{126}) \times 2^{-126}$$



Denorms: Gradual Underflow (2/3)

- Solution: **DENORM**alized number
 - no leading 1 in **mantissa**!!
 - One **implicit exponent** for all denorms:
the smallest norm step size, **2^{-126}**

31	30		23	22		1	0
s		0000 0000		xxx...xxx			

$$(-1)^s \times (0.x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}x_{11}x_{12}x_{13}x_{14}x_{15}x_{16}x_{17}x_{18}x_{19}x_{20}x_{21}x_{22}x_{23}x_{24}x_{25}x_{26}x_{27}x_{28}x_{29}x_{30}x_{31}) \times 2^{-126}$$

Smallest **denormalized** number:

0	0000 0000	000...001
---	-----------	-----------

$$2^{-149}$$

Smallest **normalized** number:

0	0000 000 <u>1</u>	000...000
---	-------------------	-----------

$$2^{-126}$$

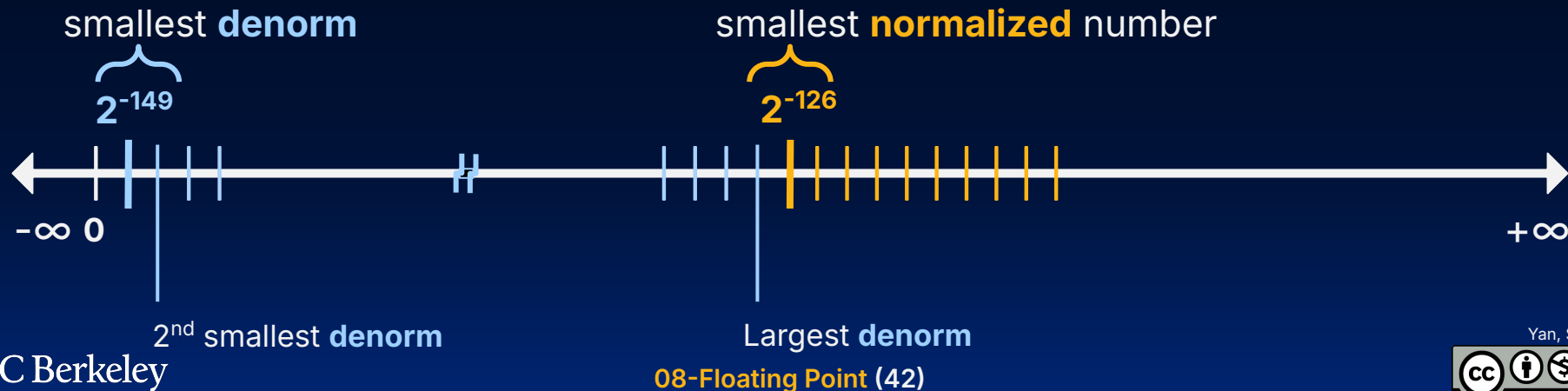


Denorms: Gradual Underflow (3/3)

At home: Convince yourself that all denorms have the same step size 2^{-149} .

Denormalized number

$$(-1)^s \times (0.x \dots x_2) \times 2^{-126}$$





WELCOME TO
THE SECRET
ROBOT INTERNET

EARLIER...

Prove you are human:

$0.1 + 0.2 = ?$

0.300000000000000004

www.smbc-comics.com/comic/2013-06-05

(double) 0.3:

0.299999999999999999999999

(double) 0.1 + (double) 0.2:

0.30000000000000004

08-Floating Point (43)